RUST

30-Day Intermediate Production Engineering Program

2 hours a day | 60 focused hours | Build, test, ship, and operate a real service

Designed for you You already know much of the syntax. This plan concentrates on ownership reasoning, API design, error models, concurrency, testing, observability, delivery, and maintenance.

Capstone: production-style Task API using Axum, Tokio, Serde, SQLx, tracing, tests, Docker, CI, migrations, and operational runbooks.

Edition: September 2026

How to Use This Program

This is a deliberate-practice program, not a reading list. Every day has a mental model, a focused build, verification, and a short reflection. Keep one Git repository for the entire month and commit daily.

Daily 120-minute rhythm 15 min recall and review; 35 min concept study; 55 min implementation; 10 min tests and linting; 5 min engineering log.

Outcome by Day 30

- Explain ownership, borrowing, lifetimes, trait bounds, dynamic dispatch, interior mutability, Send, Sync, and pinning at an intermediate level.
- Design maintainable crates and workspaces with explicit domain boundaries and dependency direction.
- Build an asynchronous HTTP service with structured errors, validation, persistence, observability, and graceful shutdown.
- Use unit, integration, property-oriented, and behavior tests appropriately.
- Create CI quality gates, a multi-stage container image, database migrations, configuration, release notes, and a rollback runbook.
- Review unfamiliar Rust code using compiler diagnostics, documentation, Clippy, and focused experiments.

Rules of Practice

- Type every example instead of copying blindly.
- Predict compiler errors before compiling.
- Keep warnings at zero. Run cargo fmt, cargo clippy, and cargo test daily.
- Prefer small commits with intent-revealing messages.
- Record one mistake, one insight, and one next question each day.
- Do not optimize until measurement shows a bottleneck.

Toolchain Setup

```
rustup update stable
rustup component add rustfmt clippy
cargo install cargo-watch
rustc --version
cargo --version
cargo clippy --version
```

Recommended editor: VS Code with rust-analyzer, or another editor with Language Server Protocol support. Use the stable toolchain for the program. Pin the toolchain in production repositories when reproducibility matters.

Baseline Assessment

Before Day 1, implement a small command-line parser in 30 minutes. It should read lines, parse a command enum, return Result instead of panicking, and include two tests. Save it as baseline/. Repeat on Day 30 and compare design quality, errors, tests, and clarity.

30-Day Roadmap at a Glance

Phase	Primary focus	Evidence
Week 1	Ownership and type-system depth	Value flow and invariants
Week 2	Crate design, errors, tests, tooling	Maintainable engineering
Week 3	Async, concurrency, networking	Correct services under load
Week 4	Production service and delivery	Build, ship, observe, maintain
Days 29-30	Hardening, review, demonstration	Operational confidence

Capstone Architecture

A layered monolith keeps the learning surface realistic without introducing unnecessary distributed-systems complexity.

```
Client
  -> HTTP routes / extractors
      -> application services
          -> domain types and rules
              -> repository trait
                  -> PostgreSQL adapter

Cross-cutting: configuration, tracing, metrics, error mapping,
graceful shutdown, migrations, CI, container image, runbook
```

Repository Shape

```
rust-production-lab/
  Cargo.toml                 # workspace
  crates/
    domain/                  # entities, value objects, repository ports
    app/                     # use cases and orchestration
    api/                     # Axum transport and binary
  migrations/
  tests/
  Dockerfile
  compose.yaml
  .github/workflows/ci.yml
  README.md
  docs/adr/0001-architecture.md
  docs/runbook.md
```

Day 1: Ownership as a resource model

FOCUS Moves, copies, drops, scopes, RAII, stack/heap distinction

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Build a resource-tracing type with Drop; predict and verify drop order.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Ownership makes resource lifetime explicit. A value has one owner; moving transfers responsibility; Drop provides deterministic cleanup. Think about who releases the resource, not only where bytes live.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Moves, copies, drops, scopes, RAII, stack/heap distinction.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Build a resource-tracing type with Drop; predict and verify drop order. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Explain why a moved value is unusable and when cloning hides a design problem.

Review prompts

- Explain ownership as a resource model in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Illustrative code

```
struct Trace(&'static str);
impl Drop for Trace {
    fn drop(&mut self) { println!("drop {}", self.0); }
}
fn main() {
    let outer = Trace("outer");
    { let inner = Trace("inner"); println!("work"); }
    println!("done");
}
```

Day 2: Borrowing and aliasing rules

FOCUS Shared vs mutable references, reborrowing, non-lexical lifetimes

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Refactor a vector-processing function to avoid clones and shorten borrow scopes.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Borrowing grants temporary access without transferring ownership. The key rule is controlled aliasing: many readers or one writer for the relevant period.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Shared vs mutable references, reborrowing, non-lexical lifetimes.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Refactor a vector-processing function to avoid clones and shorten borrow scopes. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK No unnecessary clone in the final implementation.

Review prompts

- Explain borrowing and aliasing rules in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 3: Lifetimes as relationships

FOCUS Elision, explicit relationships, structs holding references, static

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Implement a parser that returns slices tied to its input; write invalid variants and study diagnostics.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

A lifetime annotation describes a relationship the compiler must verify. It does not keep data alive and is rarely a request to make a value live longer.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Elision, explicit relationships, structs holding references, static.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Implement a parser that returns slices tied to its input; write invalid variants and study diagnostics. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Can describe what each lifetime connects without saying it extends a value.

Review prompts

- Explain lifetimes as relationships in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?

- What would fail in production if today's work were implemented carelessly?

Illustrative code

```
fn field<'a>(line: &'a str, key: &str) -> Option<&'a str> {
    line.split_once('=')
        .filter(|(k, _)| k.trim() == key)
        .map(|(_, v)| v.trim())
}
```

Day 4: Enums, pattern matching, invariants

FOCUS Algebraic data types, exhaustive matches, Option, state modeling

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Model an order workflow so invalid transitions return typed errors.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Enums let the type system express alternatives. Pair them with newtypes and private fields to centralize validation and prevent partially valid objects.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Algebraic data types, exhaustive matches, Option, state modeling.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Model an order workflow so invalid transitions return typed errors. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Illegal states are difficult or impossible to represent.

Review prompts

- Explain enums, pattern matching, invariants in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 5: Traits and generic design

FOCUS Trait contracts, bounds, associated types, blanket implementations

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Create a Repository trait and two implementations: in-memory and logging decorator.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

A trait is a behavioral contract. Put traits where the abstraction is consumed when that keeps dependency direction clean; avoid traits created only to imitate another language.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Trait contracts, bounds, associated types, blanket implementations.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Create a Repository trait and two implementations: in-memory and logging decorator. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Callers depend on behavior, not storage details.

Review prompts

- Explain traits and generic design in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?

- What would fail in production if today's work were implemented carelessly?

Illustrative code

```
pub trait TaskRepository: Send + Sync {
    fn get(&self, id: TaskId) -> Result<Option<Task>, RepoError>;
    fn save(&self, task: &Task) -> Result<(), RepoError>;
}
```

Day 6: Dispatch and object safety

FOCUS Monomorphization, impl Trait, dyn Trait, vtables, object safety

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Benchmark-free comparison: implement static and dynamic strategy selection; inspect ergonomics.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Generics usually produce specialized machine code; trait objects trade some optimization and static knowledge for runtime flexibility and heterogeneous collections.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Monomorphization, impl Trait, dyn Trait, vtables, object safety.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Benchmark-free comparison: implement static and dynamic strategy selection; inspect ergonomics. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Can justify the boundary where dynamic dispatch is useful.

Review prompts

- Explain dispatch and object safety in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?

- What would fail in production if today's work were implemented carelessly?

Day 7: Smart pointers and interior mutability

FOCUS Box, Rc, Arc, Cell, RefCell, Mutex, ownership graphs

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Build a tree with Rc/Weak and demonstrate why Weak prevents a reference cycle.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Smart pointers encode ownership policies. Box owns one heap allocation, Rc shares within one thread, Arc shares across threads, and interior mutability moves checks to runtime or locking.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Box, Rc, Arc, Cell, RefCell, Mutex, ownership graphs.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Build a tree with Rc/Weak and demonstrate why Weak prevents a reference cycle. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Select pointer types by ownership and thread-safety requirements.

Review prompts

- Explain smart pointers and interior mutability in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 8: Error design

FOCUS Result, ?, error sources, domain vs infrastructure errors, panic policy

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Create a layered error model and map low-level errors while preserving causes.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Errors are part of an API. Domain errors should be stable and meaningful; infrastructure errors should preserve their source but not leak implementation details to clients.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Result, ?, error sources, domain vs infrastructure errors, panic policy.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Create a layered error model and map low-level errors while preserving causes. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK No unwrap in normal request paths.

Review prompts

- Explain error design in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Illustrative code

```
#[derive(Debug, thiserror::Error)]
pub enum AppError {
    #[error("task {0} was not found")]
    NotFound(TaskId),
    #[error("invalid task: {0}")]
    Invalid(String),
    #[error(transparent)]
    Repository(#[from] RepoError),
}
```

Day 9: Modules, crates, workspaces

FOCUS Visibility, API surface, package layout, dependency direction

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Convert exercises into a workspace with domain, app, and api crates.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

A crate boundary is both an architectural and compilation boundary. Keep the public surface small and dependency arrows pointing inward toward stable rules.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Visibility, API surface, package layout, dependency direction.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Convert exercises into a workspace with domain, app, and api crates. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK cargo check --workspace passes and domain has no web/database dependency.

Review prompts

- Explain modules, crates, workspaces in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 10: Collections, iterators, closures

FOCUS Iterator adaptors, ownership in closures, allocation awareness

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Write an analytics pipeline first imperatively, then with iterators; compare readability.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Iterators are lazy transformation pipelines. Ownership of captured values and whether adaptors borrow, mutate, or consume are more important than clever chaining.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Iterator adaptors, ownership in closures, allocation awareness.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Write an analytics pipeline first imperatively, then with iterators; compare readability. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK No collect unless materialization is actually needed.

Review prompts

- Explain collections, iterators, closures in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 11: Testing strategy

FOCUS Unit, integration, doc tests, fakes, test boundaries

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Test domain transitions and repository behavior; add one documentation test.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Tests should protect observable behavior and important invariants. Choose the smallest test scope that gives credible evidence.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Unit, integration, doc tests, fakes, test boundaries.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Test domain transitions and repository behavior; add one documentation test. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Tests describe behavior and avoid private implementation coupling.

Review prompts

- Explain testing strategy in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Illustrative code

```
#[test]
fn completing_an_open_task_changes_state() {
    let mut task = Task::new("learn ownership").unwrap();
    task.complete().unwrap();
    assert!(task.is_complete());
}
```

Day 12: Property thinking and fuzz mindset

FOCUS Invariants, generated cases, shrinking concepts, parsing defenses

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Add proptest cases for parser round trips and malformed inputs.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Examples check selected cases; properties check relationships that should remain true across a broad input space.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Invariants, generated cases, shrinking concepts, parsing defenses.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Add proptest cases for parser round trips and malformed inputs. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK A stated invariant is checked across many generated inputs.

Review prompts

- Explain property thinking and fuzz mindset in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 13: Tooling and code quality

FOCUS rustfmt, Clippy, cargo check, docs, feature flags, MSRV concept

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Create local quality script and deny warnings in CI only.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

The compiler is a design partner. Fast check-feedback loops, automatic formatting, lints, and documentation are part of production engineering, not polish added later.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: rustfmt, Clippy, cargo check, docs, feature flags, MSRV concept.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Create local quality script and deny warnings in CI only. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK One command reproduces the quality gate.

Review prompts

- Explain tooling and code quality in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 14: Unsafe Rust literacy

FOCUS Unsafe superpowers, safety invariants, raw pointers, FFI boundaries

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Read and annotate a tiny unsafe wrapper; write a SAFETY comment and safe API.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Unsafe disables selected compiler checks, not the obligation to be correct. A safe wrapper must state and maintain every invariant required by its unsafe operations.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Unsafe superpowers, safety invariants, raw pointers, FFI boundaries.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Read and annotate a tiny unsafe wrapper; write a SAFETY comment and safe API. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Can review unsafe code without casually introducing it.

Review prompts

- Explain unsafe rust literacy in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 15: Concurrency foundations

FOCUS Threads, move closures, channels, Arc<Mutex<T>>, Send and Sync

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Implement producer-worker aggregation with channels and clean shutdown.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Rust prevents data races through ownership and Send/Sync constraints, but deadlocks, starvation, and logical races remain possible.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Threads, move closures, channels, Arc<Mutex<T>>, Send and Sync.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Implement producer-worker aggregation with channels and clean shutdown. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK No data race, no lock held during slow work.

Review prompts

- Explain concurrency foundations in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Illustrative code

```
let (tx, rx) = std::sync::mpsc::sync_channel(64);
for item in work { tx.send(item)?; }
drop(tx); // receivers can observe completion
for handle in workers { handle.join().expect("worker panicked"); }
```

Day 16: Async mental model

FOCUS Future, poll, wake, executor, async state machines

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Implement concurrent delays and compare join with sequential await.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

An async function creates a Future. Progress occurs when an executor polls it; awaiting yields when pending so another task can run.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Future, poll, wake, executor, async state machines.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Implement concurrent delays and compare join with sequential await. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Can explain why async is concurrency, not automatically parallelism.

Review prompts

- Explain async mental model in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Illustrative code

```
let (profile, permissions) = tokio::join!(
    load_profile(user_id),
    load_permissions(user_id),
);
```

Day 17: Tokio tasks and cancellation

FOCUS spawn, JoinHandle, select, timeout, cancellation safety

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Build a task supervisor with timeout and cancellation signal.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Spawning creates lifecycle responsibility. Structured ownership, cancellation, timeouts, and error observation prevent detached work from becoming operational debt.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: spawn, JoinHandle, select, timeout, cancellation safety.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Build a task supervisor with timeout and cancellation signal. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK All spawned tasks have an owner and shutdown path.

Review prompts

- Explain tokio tasks and cancellation in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Illustrative code

```
tokio::select! {
    result = worker() => result?,
    _ = shutdown.cancelled() => tracing::info!("cancelled"),
    _ = tokio::time::sleep(timeout) => return Err(AppError::Timeout),
}
```

Day 18: Async communication and backpressure

FOCUS mpsc, oneshot, watch, broadcast, bounded queues

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Build a bounded work queue and define overload behavior.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

A bounded queue turns capacity into an explicit design decision. Backpressure is often safer than unlimited memory growth.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: mpsc, oneshot, watch, broadcast, bounded queues.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Build a bounded work queue and define overload behavior. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Producer behavior under a full queue is explicit.

Review prompts

- Explain async communication and backpressure in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 19: Shared state in async services

FOCUS Async mutexes, lock scope, actor pattern, blocking boundaries

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Compare shared mutex state with a single-owner actor.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Minimize shared mutable state. When one task owns state and receives messages, invariants and mutation order can become easier to reason about.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Async mutexes, lock scope, actor pattern, blocking boundaries.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Compare shared mutex state with a single-owner actor. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK No blocking I/O on executor worker threads.

Review prompts

- Explain shared state in async services in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 20: HTTP service foundations

FOCUS Axum Router, handlers, extractors, state, JSON, status codes

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Create health and task CRUD routes backed by in-memory repository.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

The transport layer should parse and validate input, invoke an application use case, and translate outcomes. Domain logic should not depend on HTTP.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Axum Router, handlers, extractors, state, JSON, status codes.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Create health and task CRUD routes backed by in-memory repository. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK API returns correct codes and validated JSON.

Review prompts

- Explain http service foundations in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Illustrative code

```
async fn create_task(
    State(app): State<AppState>,
    Json(input): Json<CreateTask>,
) -> Result<(StatusCode, Json<TaskView>), ApiError> {
    let task = app.service.create(input).await?;
    Ok((StatusCode::CREATED, Json(task.into())))
}
```

Day 21: Persistence and migrations

FOCUS SQLx concepts, pools, transactions, migrations, repository adapter

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Add PostgreSQL repository and migration for tasks.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

A repository adapter translates between durable storage and domain models. Transactions define atomic business boundaries, not merely convenient batches of SQL.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: SQLx concepts, pools, transactions, migrations, repository adapter.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Add PostgreSQL repository and migration for tasks. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Integration test proves create/read across database boundary.

Review prompts

- Explain persistence and migrations in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 22: Configuration and secrets

FOCUS Environment config, validation, defaults, secret handling

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Create typed AppConfig and fail fast on invalid values.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Configuration is input. Parse it once into typed values, validate at startup, and keep secrets out of source, images, diagnostics, and ordinary logs.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Environment config, validation, defaults, secret handling.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Create typed AppConfig and fail fast on invalid values. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK No secret committed or printed in logs.

Review prompts

- Explain configuration and secrets in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 23: Observability

FOCUS Structured tracing, correlation IDs, latency, metrics, health/readiness

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Instrument requests and repository calls with spans and fields.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Observability should answer what failed, where, for whom, and how long it took. Prefer structured fields and spans over prose-only logs.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Structured tracing, correlation IDs, latency, metrics, health/readiness.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Instrument requests and repository calls with spans and fields. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK A failed request can be followed through logs by request ID.

Review prompts

- Explain observability in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Illustrative code

```
#[tracing::instrument(skip(repo), fields(task_id = %id))]
async fn load_task(repo: &dyn TaskRepository, id: TaskId)
    -> Result<Task, AppError>
{
    repo.get(id).await?.ok_or(AppError::NotFound(id))
}
```

Day 24: Resilience and graceful shutdown

FOCUS Timeouts, retries, idempotency, signal handling, draining

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Add timeout middleware, SIGTERM handling, and bounded shutdown.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Resilience is bounded behavior during failure. Timeouts cap waiting, retries require safe conditions, and graceful shutdown coordinates lifecycle transitions.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Timeouts, retries, idempotency, signal handling, draining.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Add timeout middleware, SIGTERM handling, and bounded shutdown. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Service stops accepting work and completes or cancels predictably.

Review prompts

- Explain resilience and graceful shutdown in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Illustrative code

```
let listener = tokio::net::TcpListener::bind(addr).await?;
axum::serve(listener, app)
    .with_graceful_shutdown(shutdown_signal())
    .await?;
```

Day 25: Security and dependency hygiene

FOCUS Input validation, least privilege, supply-chain review, unsafe audit

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Add size limits, validation, dependency audit step, and non-root container user.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Security is continuous constraint design: validate at boundaries, minimize privileges, limit resources, manage dependencies, and avoid exposing sensitive data.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Input validation, least privilege, supply-chain review, unsafe audit.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Add size limits, validation, dependency audit step, and non-root container user. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Threat checklist is documented with concrete controls.

Review prompts

- Explain security and dependency hygiene in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 26: Performance measurement

FOCUS Release profiles, allocation awareness, criterion concepts, flamegraphs

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Benchmark one domain operation and remove one measured inefficiency.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Performance work begins with a workload and measurement. Release builds, allocation patterns, lock contention, and I/O usually matter more than line-level cleverness.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Release profiles, allocation awareness, criterion concepts, flamegraphs.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Benchmark one domain operation and remove one measured inefficiency. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Optimization has before/after evidence.

Review prompts

- Explain performance measurement in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 27: Containers and reproducible builds

FOCUS Multi-stage builds, caching, small runtime image, health checks

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Create Dockerfile and compose stack for API and PostgreSQL.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

A production image should contain the runtime artifact and essential certificates/config only, run as non-root, and be reproducible from versioned inputs.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Multi-stage builds, caching, small runtime image, health checks.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Create Dockerfile and compose stack for API and PostgreSQL. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Fresh machine can build and start the system from documented commands.

Review prompts

- Explain containers and reproducible builds in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Illustrative code

```
FROM rust:1-slim AS build
WORKDIR /src
COPY . .
RUN cargo build --locked --release -p api

FROM debian:stable-slim
RUN useradd --system --uid 10001 app
COPY --from=build /src/target/release/api /usr/local/bin/api
USER app
ENTRYPOINT ["/usr/local/bin/api"]
```

Day 28: CI/CD and releases

FOCUS Quality gates, test layers, artifacts, versioning, rollback

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Create GitHub Actions CI and a release workflow outline.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

CI creates repeatable evidence; deployment is a controlled promotion of an immutable artifact. Keep build and deploy separable, with an understood rollback path.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Quality gates, test layers, artifacts, versioning, rollback.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Create GitHub Actions CI and a release workflow outline. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Pull request fails on format, lint, test, or build problems.

Review prompts

- Explain ci/cd and releases in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Illustrative code

```
- name: Quality gates
  run: |
    cargo fmt --all -- --check
    cargo clippy --workspace --all-targets -- -D warnings
    cargo test --workspace --all-features
    cargo build --workspace --release --locked
```

Day 29: Operational hardening

FOCUS Runbooks, migrations, backup assumptions, SLOs, incident thinking

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Run failure drills: bad config, unavailable DB, slow request, shutdown during traffic.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Production readiness includes diagnosis and recovery. Failure drills expose missing timeouts, vague errors, unsafe migrations, and undocumented assumptions.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Runbooks, migrations, backup assumptions, SLOs, incident thinking.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Run failure drills: bad config, unavailable DB, slow request, shutdown during traffic. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Runbook tells an operator how to diagnose and recover.

Review prompts

- Explain operational hardening in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Day 30: Final review and demonstration

FOCUS Architecture defense, code review, release candidate, next plan

Time	Activity
0-15 min	Recall yesterday without notes; answer the review prompt.
15-50 min	Study the mental model; run two micro-experiments.
50-105 min	Repeat baseline, tag v0.1.0, demo API, and write retrospective.
105-115 min	Run fmt, Clippy, and tests; inspect diff.
115-120 min	Write mistake, insight, next question, and commit.

Mental model

Intermediate skill means independently navigating unfamiliar code, making justified tradeoffs, and shipping changes with tests and operational awareness.

Micro-experiments

- Create the smallest compilable example that demonstrates one rule from: Architecture defense, code review, release candidate, next plan.
- Create a deliberately failing variant. Read the full diagnostic, predict the smallest correction, then fix it without adding clone or unwrap unless justified.

Production connection

Apply today to the capstone. Repeat baseline, tag v0.1.0, demo API, and write retrospective. Keep domain rules isolated, make failure behavior explicit, and leave the repository in a deployable state.

Definition of done

EXIT CHECK Can explain tradeoffs, operate the service, and name next learning priorities.

Review prompts

- Explain final review and demonstration in your own words without syntax-first definitions.
- Which tradeoff did you make today, and what alternative did you reject?
- What would fail in production if today's work were implemented carelessly?

Capstone Implementation Guide

Build the service vertically. Each slice should include domain behavior, application orchestration, adapter behavior, endpoint mapping, tests, and observability before starting a new feature.

Domain model

```
pub struct Task {
    id: TaskId,
    title: TaskTitle,
    status: TaskStatus,
    version: u64,
}

pub enum TaskStatus { Open, Complete }

impl Task {
    pub fn complete(&mut self) -> Result<(), DomainError> {
        match self.status {
            TaskStatus::Open => { self.status = TaskStatus::Complete; self.version += 1; Ok(()) }
            TaskStatus::Complete => Err(DomainError::AlreadyComplete),
        }
    }
}
```

HTTP error mapping

```
impl IntoResponse for ApiError {
    fn into_response(self) -> Response {
        let (status, code, message) = match self.0 {
            AppError::NotFound(_) => (StatusCode::NOT_FOUND, "not_found", self.0.to_string()),
            AppError::Invalid(_) => (StatusCode::BAD_REQUEST, "invalid_request",
self.0.to_string()),
            _ => (StatusCode::INTERNAL_SERVER_ERROR, "internal", "internal error".into()),
        };
        (status, Json(json!({"code": code, "message": message}))).into_response()
    }
}
```

Testing pyramid for this project

- Many domain unit tests: transitions, validation, invariants.
- Application tests with a controllable in-memory repository.
- HTTP integration tests through the router without opening a real port.
- Database adapter tests against an ephemeral PostgreSQL instance or dedicated test database.
- A small end-to-end smoke test after deployment: health, create, read, complete.

Production checklist

☐ Pinned toolchain or documented minimum supported Rust version

☐ Cargo.lock committed for application

☐ Formatting, Clippy, tests, and release build in CI

☐ No normal-path unwrap/expect

☐ Typed startup configuration validation

☐ Structured logs with request/correlation ID

☐ Readiness verifies required dependencies

☐ Timeouts and payload limits

☐ Graceful shutdown and task ownership

☐ Forward and rollback migration plan

☐ Non-root container and minimal image

☐ Dependency and license review

☐ README quick start plus operator runbook

☐ Immutable artifact tagged with source revision

Runbook skeleton

- Service purpose and owners
- Dependencies and expected ports
- Configuration keys and secret locations
- Health and readiness interpretation
- Dashboard and log queries
- Common symptoms and diagnostic steps
- Database migration procedure
- Rollback steps and compatibility limits
- Data recovery assumptions
- Escalation and post-incident actions

Assessment Rubric

Dimension	Intermediate evidence	Score 0-3
Ownership reasoning	Explains moves/borrows/lifetimes and removes needless cloning	___
Type-driven design	Uses enums/newtypes/private fields to protect invariants	___
Error handling	Typed errors, preserved sources, stable client mapping, clear panic policy	___
Architecture	Small public APIs, inward dependencies, replaceable adapters	___
Concurrency	Task ownership, bounded queues, short lock scopes, cancellation	___
Testing	Behavioral tests at appropriate layers, credible failure cases	___
Operations	Configuration, logs, health, shutdown, migrations, runbook	___
Delivery	Repeatable CI, locked release build, container, rollback story	___

Interpretation: 0 = absent; 1 = follows examples; 2 = independently applies with minor gaps; 3 = explains tradeoffs and handles failure modes. Target at least 16/24 overall, with no zero in ownership, errors, testing, or operations.

After the 30 Days

- Contribute one documentation or test improvement to an established Rust project.
- Read one production Rust repository weekly and summarize its crate boundaries and error model.
- Choose one specialization for the next 30 days: backend services, command-line tools, embedded, systems, or WebAssembly.
- Revisit unsafe, pinning, macros, and advanced lifetimes only when a real use case creates context.

Curated References

Use official documentation as the default authority. Crate APIs evolve, so verify examples against the version locked in Cargo.lock.

The Rust Programming Language
Core language mental models and guided examples.
https://doc.rust-lang.org/book/

Rust Reference
Precise language behavior when the Book is not detailed enough.
https://doc.rust-lang.org/reference/

The Cargo Book
Packages, workspaces, features, profiles, publishing, and commands.
https://doc.rust-lang.org/cargo/

Rust Standard Library
Types, traits, modules, and extensive examples.
https://doc.rust-lang.org/std/

Rust API Guidelines
Checklist for idiomatic public library APIs.
https://rust-lang.github.io/api-guidelines/

Rustonomicon
Advanced unsafe Rust concepts. Read for literacy, not as a beginner cookbook.
https://doc.rust-lang.org/nomicon/

Clippy Documentation
Lint configuration and rationale.
https://doc.rust-lang.org/clippy/

Tokio Documentation
Async runtime, tasks, synchronization, I/O, and runtime behavior.
https://docs.rs/tokio/latest/tokio/

Axum Documentation
Routing, extractors, responses, state, middleware integration.
https://docs.rs/axum/latest/axum/

Serde
Serialization and deserialization patterns.
https://serde.rs/

SQLx
Async SQL toolkit and compile-time checked query options.
https://docs.rs/sqlx/latest/sqlx/

tracing
Structured, span-based diagnostics for async systems.
https://docs.rs/tracing/latest/tracing/

Daily Engineering Log Template

Date / Day: _____

What I built: _____

Prediction that was wrong:

_____

Compiler or test feedback:

_____

Tradeoff made: _____

One production risk considered:

_____

Tomorrow's first action: _____